

Evolutionary inversion of class distribution in overlapping areas for multi-class imbalanced learning

Everlandio R.Q. Fernandes^{a,b,*}, Andre C.P.L.F. de Carvalho^a

^a University of São Paulo, São Carlos SP, Brazil

^b Sidia Institute of Science and Technology, Manaus AM, Brazil



ARTICLE INFO

Article history:

Received 27 March 2018

Revised 27 February 2019

Accepted 26 April 2019

Available online 26 April 2019

Keywords:

Multi-Class imbalanced learning

Ensemble of classifiers

Evolutionary algorithms

ABSTRACT

Inductive learning from multi-class and imbalanced datasets is one of the main challenges for machine learning. Most machine learning algorithms have their predictive performance negatively affected by imbalanced data. Although several techniques have been proposed to deal with this difficulty, they are usually restricted to binary classification datasets. Thus, one of the research challenges in this area is how to deal with imbalanced multiclass classification datasets. This challenge become more difficult when classes containing fewer instances are located in overlapping regions of the data attribute space. In fact, several studies have indicated that the degree of class overlapping has a higher effect on predictive performance than the global class imbalance ratio. This paper proposes a novel evolutionary ensemble-based method for multi-class imbalanced learning called the evolutionary inversion of class distribution in overlapping areas for multi-class imbalanced learning (EVINCI). EVINCI uses a multiobjective evolutionary algorithm (MOEA) to evolve a set of samples taken from an imbalanced dataset. It selectively reduces the concentration of less representative instances of the majority classes in the overlapping areas while selecting samples that produce more accurate models. In experiments performed to evaluate its predictive accuracy, EVINCI was superior to state-of-the-art ensemble-based methods for imbalanced learning.

© 2019 Published by Elsevier Inc.

1. Introduction

Today, many data classification tasks involve imbalanced datasets, in which at least one class is underrepresented. This situation is found in many real-world problems, such as data analysis of fraudulent credit card transactions, disease diagnosis, image analysis of defective parts on production lines, and ecology, to name a few. Consequently, the development and investigation of techniques that can perform effective data classification in imbalanced datasets are currently one of the most compelling research issues in data mining and machine learning [32].

Most classical classification algorithms have difficulties in dealing with imbalanced datasets. These difficulties occur because in order to improve the overall predictive accuracy of a training data subset, they usually induce classification models that tend to give less consideration to classes with few examples (minority classes). This situation becomes even more challenging when objects from minority classes are situated in overlapping regions of the data attribute space. In fact, several

* Corresponding author.

E-mail address: everlandio@usp.br (E.R.Q. Fernandes).

studies [19,32,34] have indicated that class distribution is not primarily responsible for hindering classifier performance, but rather it is the degree of overlap between the dataset classes.

In particular, [19] presented an interesting study on this subject. The authors proposed two different frameworks that focus on the performance of the K -NN classification algorithm. In the first framework, they investigate the situation in which the imbalance ratio in the overlap region is similar to the overall imbalance ratio. In the second, the imbalance ratio in the overlapping areas is inversely related to the overall imbalance ratio, that is, the minority class is locally denser in the overlapping regions. Their experimental results seemed to indicate that the behavior of the K -NN is more dependent on changes in the imbalance ratio in the overlapping region than on changes in the size of the overlapping area. Their results also indicated that the proportion of the local imbalance ratio and the size of the overlapping region are more important than the relationship with the global imbalance ratio. [32] indicate that, in the case of an overlapping region, most classification algorithms are not only unable to correctly discriminate between classes, but they also favor the majority classes, which leads to low overall classification accuracy.

Although many studies highlight the problem of dominance of a class in the overlapping region, its treatment has received little attention in the imbalanced learning literature, as discussed in a recent study about data irregularities in classification tasks [10].

Various measures for estimating the complexity of the class boundary and its overlapping rates have been investigated over the years. In particular, [25] proposed and evaluated several measures that characterize the difficulty of a classification problem, focusing on the geometric complexity of the class boundary. One of these measures is based on the test proposed by [17], which verifies whether two samples, for example, two datasets collected at different times, come from the same data distribution. Ho and Basu used this test, which they called the N1 measure, to decide whether instances from different classes form separable distributions.

The N1 measure initially generates a minimum spanning tree (MST) that connects all instances of a dataset, taking into account the Euclidean distance and ignoring the class associated with each instance. Next, N1 counts the number of instances connected by an edge in the MST and belonging to different classes. These instances are considered to be close to the class boundary. As this measure was originally designed for binary datasets, it uses the ratio of the sum of these points to the total points in the dataset as the measure of complexity that estimates the separability of classes, that is, the fraction of instances in the dataset that lies at the class boundary. High N1 values indicate the need for a more complex separation boundary for the dataset.

When classifying multi-class imbalanced datasets, it would be interesting to verify the value of this ratio for all classes and, in particular, between the majority and minority classes. By doing so, this measure could be used to optimize the sampling of the dataset so that the overlapping areas present a higher concentration of examples from the minority classes, thereby increasing their visibility to the classification algorithm. However, even though the overlapping regions have a higher concentration of minority classes, the resulting classifier may have low overall accuracy since relevant instances of the majority classes could be eliminated, thus impeding their recognition. That is, increasing the accuracy of some classes can lower the accuracy of others.

One possible solution to the situation described above is to combine ensembles of classifiers and multiobjective evolutionary algorithms (MOEAs). In contrast to traditional machine learning methodologies that construct a single hypothesis (model) of the training dataset, ensemble learning techniques induce a set of hypotheses and combine them through some consensus method or operator. An essential characteristic of ensembles of classifiers is their generalization power, which is greater than that of the classifiers composing the ensemble (base classifiers), as formally presented in [43]. The MOEAs can adequately manage conflicting objectives in the learning process by simultaneously evolving a set of solutions into two or more objectives without needing to impose preferences on the objectives.

Using this context, in this paper, we propose a new ensemble-based method for multi-class imbalanced learning, which we call the evolutionary inversion of class distribution for imbalanced learning (EVINCI). Using a MOEA, EVINCI optimizes a set of samples taken from the training dataset so that they present a higher concentration of instances of the minority classes in the overlapping areas, thereby making these classes more evident. We developed an extension of the N1 complexity measure, called N1byClass, for EVINCI to estimate the overlap percentage of each pair of classes. With the support of the N1byClass measure and the improved accuracy of the model induced by the samples, EVINCI selectively reduces the concentration of less representative instances of the majority classes in the overlapping areas while also selecting the samples that produce more accurate models.

To increase its generalization power and reduce the information losses resulting from the selection process, the EVINCI classification system consists of an ensemble of classifiers, in which each base classifier is induced by a different optimized sample. Moreover, processing of the proposed method incorporates two rules to promote the diversity of the classifiers generated by its evolutionary process, based on the necessary condition for building an effective ensemble of classifiers [12]. First, it eliminates samples generated with a high degree of similarity. Then, the measure of classifier diversity, pairwise failure crediting (PFC) [8], resolves any tie issues in the selection process.

The remainder of this paper is organized as follows. In Section 2, we discuss the problem of classifying a multi-class imbalanced dataset and the classical approaches that have been proposed in the literature to address this problem. In Section 3, we present the use of an ensemble of classifiers to solve the classification problem in imbalanced datasets. In Section 4, we propose our solution whereby we change the complexity measure N1 for use in the multi-class datasets domain. In Section 5, we present a more detailed description of the proposed method. We performed experiments on 22 datasets with

different imbalance ratios and numbers of classes ranging from 2 to 18. We describe the experiments and discuss the results we obtained in [Section 6](#). Finally, in [Section 7](#), we present our conclusions.

2. Imbalanced dataset methods and issues

Generating of classification models from imbalanced datasets has been extensively addressed in the machine learning literature over the last 20 years. However, most of the proposed techniques were designed and tested for binary dataset scenarios, i.e., datasets with two classes. In this case, researchers focus on the correct classification of the class with fewer examples (minority class), since the classifier usually tends to choose the majority class by default. Unfortunately, when a multi-class dataset is presented, the solutions proposed in the literature are not directly applicable or achieve below-expected performance [\[15\]](#).

Class decomposition is a commonly applied solution to multi-class problems, whereby a multi-class problem is transformed into a set of sub-problems, each with two classes. The most common application of this technique is as follows: given a dataset with more than two classes, one class is chosen as the positive class, and all the other classes are labeled as contrary to the positive, i.e., negative. This new dataset labeling is used to induce a binary classifier. The process is repeated so that in each round a different class is chosen as positive [\[37\]](#). For example, for a dataset with five classes, five binary classifiers will be generated. This technique is known as one-against-all or one-vs-others. However, in their study, [\[45\]](#) examined many issues related to the classification of multi-class imbalanced datasets. Regarding the decomposition class, their study results indicated that this methodology provided no advantages for multi-class imbalance learning, and even made the generated sub-problems more imbalanced.

When considering imbalanced binary datasets, the solutions proposed in the literature can be divided into two groups: the data level and algorithm level. The first group, which is most popular in the literature, preprocesses the dataset to be presented to the classification algorithm. The goal of these methods is to rebalance the classes by resampling the data space. This is mainly achieved by undersampling the majority classes, oversampling the minority classes, or some combination of both. Thus, the expected result is to contain any system bias for the majority class that is due to the different class distribution. The main advantage of these techniques is that they are independent of the classification algorithms used in the next phase.

Typically, undersampling only eliminates examples of the majority classes. When undersampling is performed randomly (random undersampling (RUS)), there may be a loss of relevant information from the classes that have been reduced. Directed or informative undersampling attempts to work around this problem by detecting and eliminating a less significant fraction of the data. This is the strategy used in the one-sided selection (OSS) technique [\[29\]](#), which attempts to remove redundant and/or noisy instances from the majority class lying close to the boundary. Border instances are detected by applying Tomek links and instances far from the decision boundary (redundant instances) are discovered using the condensed nearest neighbor (CNN) rule [\[23\]](#). Eliminating examples from the majority class lying close to the separation boundary is also performed by the majority undersampling technique (MUTE) [\[7\]](#), which defines security levels for each instance from the majority class and uses them to determine the need for undersampling.

In oversampling methods, elements of the minority class are replicated or generated synthetically until the size of the minority class is close or equal to that of the other class. In random oversampling (ROS), minority class instances are randomly selected and replicated. The synthetic minority over-sampling technique (SMOTE) [\[9\]](#), another example of an algorithm that uses oversampling, uses data interpolation to synthetically generate new instances of the minority classes. To generate a new example, SMOTE considers the feature space and then selects an example from the minority class and finds its k -nearest neighbors that share the same class. Then, it synthetically generates new instances along line segments that join any/all of its k -nearest neighbors.

Depending on how the examples are generated, oversampling techniques can increase the overlap between classes. Some methods have been proposed to minimize this drawback, including the modified synthetic minority oversampling technique (MSMOTE) [\[27\]](#) and adaptive synthetic sampling (ADASYN) [\[24\]](#). In [\[38\]](#), the authors propose another extension to the SMOTE method, the SMOTE-IPF. In this method, the ensemble-based filter IPF [\[28\]](#) is applied after generating new cases by SMOTE, so that, through an iterative process, select and remove those examples considered as noise. Another aspect to be addressed is the tendency of instance replication to increase the computational cost of the learning process [\[41\]](#) and generation data that may not make sense in the investigated problem.

The second group of solutions proposed to address imbalanced binary datasets are those in the algorithm group. These solutions are based on the adaptation of some existing classification algorithm to alleviate the bias of these algorithms toward the majority class. There are three main categories in this group – those based on recognition, cost-sensitive methods, and ensemble-based methods.

Recognition-based methods take the form of one-class learners. This is an extreme case in which only examples from one class are used to induce the classification model, usually instances from the minority class. The one-class SVM method [\[39\]](#) is recognition-based in that it considers only the minority class during the learning process to determine the class of interest. It infers the properties of minority class cases and from those can predict which examples differ from the class of interest. However, as indicated by the authors in [\[2\]](#), some classification algorithms such as decision trees, naive-Bayes, and others do not work with examples from only one class, which makes these methods unpopular and restricts them to certain learning algorithms.

Many existing classification algorithms are designed to assign equal costs to the errors made in different classes, and the modification of this criterion is the central proposal of cost-sensitive methods. These approaches include assigning different costs to incorrect predictions or developing training criteria that are more sensitive to a skewed distribution. The latter is the case in dynamic sampling (DyS) [31], which proposes to train a multilayer perceptron (MLP). At each epoch of the MLP training process, DyS uses a decision heuristic based on the current status of the MLP to decide which examples will be used to update the MLP weights. As noted by the authors in [41], solutions at the algorithm level are usually specific to a particular algorithm and/or problem. Consequently, they are only useful in certain contexts and generally require experience in the field of application and in the classification algorithms used.

In recent years, there has been an increasing use of ensembles of classifiers as a possible solution for imbalanced learning [20,35,46]. The proposed solutions are based on a combination of ensemble learning techniques and some resampling method or cost-sensitive method, or an adaptation of some existing classification algorithm. We discuss the use of ensembles of classifiers for imbalanced datasets in the next section.

3. Imbalanced ensemble learning

Ensemble methods leverage the classification power of base classifiers by combining them to form a new classifier that outperforms each of them. [12] discusses and provides an overview of why ensemble methods usually outperform single classifier methods. In the work conducted by [22], the authors proved that under specific constraints the expected error rate of an instance goes to zero as the number of base classifiers goes to infinity. To do so, the base classifiers must have an accuracy rate higher than 50% and be as diverse as possible. Two classifiers are considered to be diverse if their misclassifications are made at different instances in the same test set. Considering the techniques used to construct an ensemble of classifiers, the algorithms most often used are those based on bagging [6] and boosting [16] methods.

In the bagging method, different samples bootstrapped from the training dataset induce the set of base classifiers. Sampling is performed by replacement, and each sample has the same size and class distribution as the original training dataset. When an unknown case appears in the system, each base classifier makes a prediction, and the class with the most votes is assigned to the new instance.

Bagging-based methods that have been proposed to deal with imbalanced datasets differ mainly in the way they collect the samples that induce the base classifiers. These methods construct balanced samples from the training dataset. That is, the problem of imbalanced datasets is addressed by pre-processing the samples before inducing each classifier. As such, different sampling strategies lead to different bagging-based methods. Random oversampling bagging (OverBagging) and SMOTEBagging [44] methods perform oversampling upon each iteration of the bagging method. The OverBagging method conducts a random oversampling of the minority classes and SMOTEBagging synthetically generates new instances using the SMOTE algorithm. The UnderBagging [4] method performs a random undersampling of the majority classes before inducing the base classifiers.

The AdaBoost method [16] is the most typical algorithm in the boosting family. The complete training dataset is used to sequentially generate the base classifiers. During each iteration, examples incorrectly classified in the previous iteration are emphasized when a new classifier is being induced. In the AdaBoost method, the base classifiers produce weighted votes based on their overall accuracy. Therefore, when a new instance is presented, each base classifier gives its vote, and the class that receives more votes is assigned to the new instance.

As mentioned in [18], boosting algorithms are usually fused with cost-sensitive learning or resampling techniques, which then respectively generate cost-sensitive boosting or boosting-based ensembles. The AdaBoost algorithm is oriented toward overall accuracy and, as noted above, when the dataset is imbalanced this kind of algorithm tends to promote new instances as being in the majority class. For this reason, cost-sensitive boosting algorithms propose to change how the weights of instances are updated, prioritizing the minority class. This is the case of the AdaCost [13] method, which increases the weights of misclassified examples of minority class more aggressively but decreases the weights of correctly classified samples more conservatively. As in bagging-based methods, boosting-based methods use some re-sampling technique to deal with imbalanced datasets. This is the case in RUSBoost [40], which applies random undersampling in the original training dataset at each iteration and adjusts the distribution of the weights of instances according to the new size of the dataset.

Several methods were proposed aiming to improve the diversity and accuracy of the base classifiers simultaneously. These methods typically use some evolutionary algorithms as mechanisms for evolving a group of solutions or classifiers. This is the case in multiobjective genetic sampling (MOGASamp) [14], which builds an ensemble of classifiers by manipulating samples taken from the training dataset. In this method, each sample represents an individual in the MOEA proposed by the authors, which is being evaluated based on the classification model that it induces. The authors defined the objectives of the MOEA as the selection of samples that induce classifiers with higher accuracy and that present more significant divergence from other classifiers. In [5], the authors proposed a multiobjective genetic programming (MOGP) method that uses the accuracies of the minority and majority classes as opposing objectives in the learning process. The MOGP approach was adapted to evolve diverse solutions into an ensemble, thereby improving the general classification performance. Both the MOGASamp and MOGP methods were proposed to deal with binary datasets.

Table 1
N1byClass based on the MST shown in Fig. 1.

Class	Square	Heptagon	Circle
Square	0.4	1.0	0.2
Heptagon	1.0	1.2	0.4
Circle	0.2	0.4	0.8

4. N1byClass

As discussed earlier, Ho and Basu proposed the N1 measure of complexity, which estimates the separability of classes in binary datasets. N1 is based on the percentage of points (instances) of the dataset that are connected in a minimum spanning tree (MST) and belong to different classes. The MST can be directly applied to a range of fields, including network design, image processing, and clustering algorithms [21].

Given a dataset, a spanning tree for this dataset is a graph that contains all the instances of the dataset that are connected by a weighted and non-directed edge and that have no cycle. Thus, for a given dataset, several spanning trees with different associated costs (sum of weighted edges) can be constructed. An MST is a spanning tree whose cost is the lowest of all the possible spanning trees [21]. Considering the weights associated with each edge as a Euclidean distance, then two points connected in an MST and belonging to different classes either lie at the boundary of separation of the two classes or one of these points represents noise.

Based on this concept, our proposal of the N1byClass is to generate an MST for the dataset to connect the points by their Euclidean distance. After this step, each class of the dataset is checked for the percentage of its instances that are connected to the other classes and to itself, which generates a matrix of values. The value corresponding to N1byClass for the class i , considering the class j , is given by Eq. (1), as follows:

$$N1byClass_{\{i,j\}} = \frac{1}{n_i} \sum a_{i,j} \quad (1)$$

where n_i is the number of elements of class i and a_{ij} represents the existence of a connection between an instance of class i and an instance of class j . It must be observed that when there is a connection between instances of the same class, this connection is counted twice. However, this situation is disregarded by EVINCI, since it looks for information on areas of overlap between the minority classes and the majority classes. Section 5.1 explains how EVINCI makes use of the N1byClass matrix.

Fig. 1 shows an MST for a dataset with three classes and 15 instances. For the MST shown in Fig. 1, Table 1 shows the resulting N1byClass.

5. Proposed method

The primary objective of EVINCI is to build an ensemble of classifiers with high accuracy and generalization power for multi-class imbalance classification. These base classifiers are induced by optimized and diverse samples from imbalanced datasets. To do so, the proposed method uses an MOEA to evolve a combination of samples, guided by the class distribution in regions of overlap between majority and minority classes and by the accuracy of the classifiers induced by the samples. EVINCI resolves tie situations in the selection process by the PFC measure of classifier diversity. PFC has been shown to produce good estimates of the diversity of classifiers when applied to imbalanced learning problems [5,8,14]. The use of PFC and a mechanism to eliminate similar solutions after the crossover process promotes the creation of diverse solutions in the evolutionary process. Fig. 2 shows the workflow of the proposed method.

EVINCI uses a customized well-known MOEA, based on NSGA II [11], that uses the non-dominance rank of its objectives to select the most suitable solutions. Non-dominance rank is a common Pareto-based dominance measure that calculates the number of other solutions in a population that dominates a given solution. In Pareto dominance, a solution x_1 dominates another solution x_2 if it is no worse than x_2 in any objective, and x_1 is undoubtedly better than x_2 in at least one of them. This technique allows individuals to be ranked according to their performance on all the objectives for all individuals in the population. Therefore, a non-dominated solution will have the best fitness of 0, whereas high fitness values indicate poor-performing solutions, i.e., solutions dominated by many other solutions.

To systematically decide which classes are majority and minority in a multi-class dataset, EVINCI uses a limit based on the class distribution of the dataset. Thus, all classes having fewer elements than this value are considered to be minority. Eq. (2) shows how this limit is calculated:

$$lim = Abs \left(mean(Sc_1, Sc_2, \dots, Sc_n) - \frac{SD(Sc_1, Sc_2, \dots, Sc_n)}{2} \right) \quad (2)$$

where Sc_n represents the number of instances of the class n in the training dataset, $mean$ is the arithmetic mean, SD is the standard deviation, and Abs indicates that the absolute value of the result must be extracted.

Table 2

Basic dataset characteristics (#C: Number of Classes, #F: number of features, imbalance ratio, class distribution. minority classes indicated by Eq. (2) are in bold.).

Data Set	#Class	#F	Imb Ratio	Class Distribution
Abalone	18	8	1.0572	15: 57: 115: 259: 391: 568: 689: 634: 487: 267: 203: 126: 103: 67: 58: 42: 32: 26
Balance-scale	3	4	1.0457	49: 288: 288
Car	4	6	1.1029	384: 69: 1210: 65
Chess	18	6	1.0604	2796: 1433: 2854: 2166: 471: 198: 4553: 1712: 78: 683: 592: 390: 1985: 4194: 81: 3597: 246: 27
Contraceptive	3	9	1.1499	629: 333: 511
Dermatology	6	34	1.0334	112: 61: 72: 49: 52: 20
Dnormal	3	2	1.1962	324: 342: 84
Ecoli	5	7	1.1674	143: 77: 35: 20: 52
Ecoli2	2	7	2.8223	284: 52
Glass	4	9	1.1280	70: 76: 17: 29
New-thyroid	3	5	1.7762	150: 35: 30
Nursery	4	8	2.0267	4320: 4266: 4044: 328
Oilspill	2	49	10.9497	896: 41
Page-blocks	5	10	7.1041	4913: 329: 28: 88: 115
Penbased	10	16	1.0002	115: 114: 114: 106: 114: 106: 105: 115: 105: 106
Poker	2	10	16.3789	2050: 25
Satellite	6	36	1.0532	1533: 703: 1358: 626: 707: 1508
Shuttle	3	9	2.6304	1706: 338: 123
Thyroid	3	20	8.2745	17: 37: 666
Winequality	2	11	41.0061	880: 20
Yeast	9	8	1.1651	463: 35: 44: 51: 163: 244: 429: 20: 30
Yeast5	2	8	22.0114	1440: 44

Eq. (2), which we developed during the experimental tests of our work, fits very well with the datasets used in our experiments, but needs more validation with other datasets. We emphasize that we used Eq. (2) as a systematic form of decision-making, but the proposed method can accept the manual insertion of the definition of classes by a dataset specialist. Table 2 shows in bold the dataset classes used in the experiments that were identified as minority classes.

Next, we highlight the main contributions of EVINCI and how EVINCI differs from the other methods also proposed to deal with imbalanced learning. First, as far as the authors know, EVINCI is the first method based on the concept that models induced from samples from the original imbalanced dataset with a higher concentration of instances from the minority classes in the overlap regions can build ensembles of classifiers with higher generalization ability and accuracy. EVINCI disregards the overall imbalance rate of the dataset, focusing on the overlap areas. For such, EVINCI uses a complexity measure specifically designed to extract information regarding overlap regions in datasets, the N1byClass. This is another important contribution from this paper. It must be observed that during the evolutionary process there is no restriction to the imbalance growth of the samples.

EVINCI employs NSGA-II for sampling optimization. However, the process of the evolutionary algorithm was changed in several points to fit the desired search. First, a process was introduced to eliminate similar solutions and the crowding distance was replaced by a measure of diversity of classifiers, since it was seen to be better for the desired search, as explained in Section 5.3. In addition, EVINCI uses a strategy to maintain the best ensemble found during sample evolution. This strategy differs from most elitism strategies used in evolutionary algorithms. In the proposed strategy, the fittest individuals go from one generation to another, but the new version of elitism proposed for EVINCI maintains the best result of the interaction of the individuals throughout the generations, that is, the population that produced the most effective ensemble of classifiers. Section 5.3 explains how the process that results in the “Saved Ensemble” occurs.

5.1. Initial population and fitness

To generate a diverse initial population, random sampling is performed with different imbalance ratios up to the 1:3 limit. According to the authors of the study performed in [34], this level of imbalance causes no significant loss of accuracy in most classifiers. The individuals (samples) are represented by binary vectors with the same size as the training dataset, for which each cell represents an instance of the training dataset. The vector value 1 indicates that the instance corresponding to that position in the training dataset is present in the sample and the value 0 represents its absence. Fig. 3a shows two examples of vector representation of individuals for a dataset with 15 instances and three classes.

After generating the initial population and identifying which classes are majority and which are minority (either manually or by Eq. (2)), an N1byClass matrix is generated for each individual. The first objective of the adapted MOEA is computed as the sum of the values corresponding to the percentage of instances of the majority classes that are at the border of separation with the minority classes. This means that in the N1byClass matrix, the cells are identified by rows relating to majority classes and columns relating to minority classes. The genetic algorithm looks for solutions that have lower values of this objective, that is, samples in which the majority classes present fewer connections in the MST with minority classes.

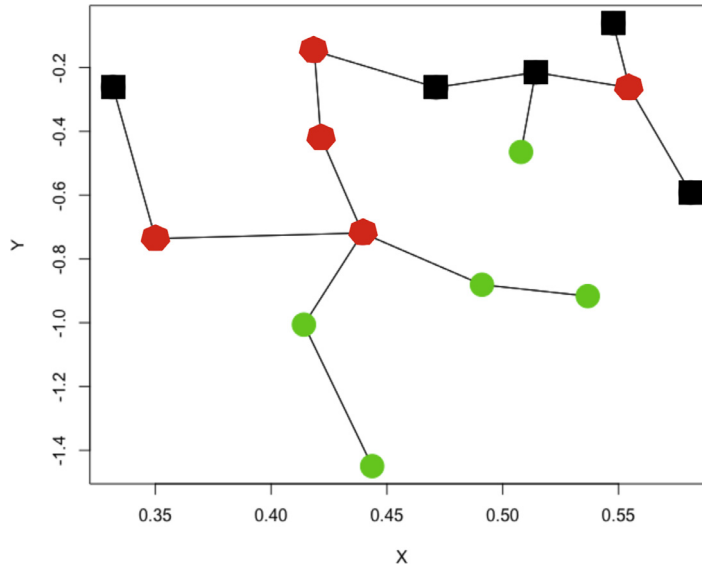


Fig. 1. Minimum spanning tree.

The second objective of the adapted MOEA is the accuracy of the classification model generated by the sample. Each individual induces a classification model and its accuracy is calculated by verifying its effectiveness in the complete training dataset. We used the G-mean [46] as the accuracy measure in our study. Since this measure determines the geometric mean of the correctness of a given classifier, low accuracy in at least one class leads to a low G-mean value, which makes it less useful in practice. Eq. (3) shows how the G-mean is calculated:

$$G - mean = \left(\prod_{i=1}^m \frac{tr_i}{n_i} \right)^{\frac{1}{m}} \quad (3)$$

where m is the number of classes, n_i is the number of examples in class i , and tr_i is the number of correctly classified examples in class i .

5.2. Selection and reproduction

The values for the above two objectives are used to generate the non-dominance rank, which represents the fitness level of each individual in the population and is used to select individuals who will participate in the breeding process. This selection is performed based on a tournament between three individuals and tie situations are decided randomly. The tournament process returns a number of pairs of individuals equal to the size of the population.

For each pair of parents, two new individuals are generated using one-point crossover [33] per class. It selects a single crossover point on both sub-vectors representing the instances of a given class, and all data beyond that point are exchanged between the two parents. After applying the process to all classes in the dataset, the sub-vectors are consolidated to form the resulting offspring. Fig. 3 presents two individuals representing samples of a dataset with 15 instances and 3 classes (Fig. 3a) and the crossover process for the class represented by black squares (Fig. 3b). The mutation occurs in 10% of the generated offspring, by inverting the bits of a randomly selected part of the vector representing an individual. The size of this random part varies between 1% and 10% of the vector size. Fig. 4 illustrates the mutation process.

After the reproduction process, the proposed method applies a mechanism to eliminate samples with a high level of similarity. During our experimental tests, we observed that similar individuals with high fitness are more likely to be selected for the breeding process to produce future generations. This increases the number of similar samples and reduces the expected diversity of the solutions. This is why the elimination mechanism is invoked to discard individuals with similarity levels greater than 85%. After this process, if the number of individuals is less than the initial population size, new reproduction and mutation processes are performed.

5.3. New generation, saved ensemble and stop criteria

All individuals in the offspring population are evaluated according to the defined objectives and are joined together with the current population to form an intermediate population. Then, the non-dominance rank is rebuilt for this intermediate population and, based on the updated rank, individuals are selected to comprise the new generation. First, individuals with

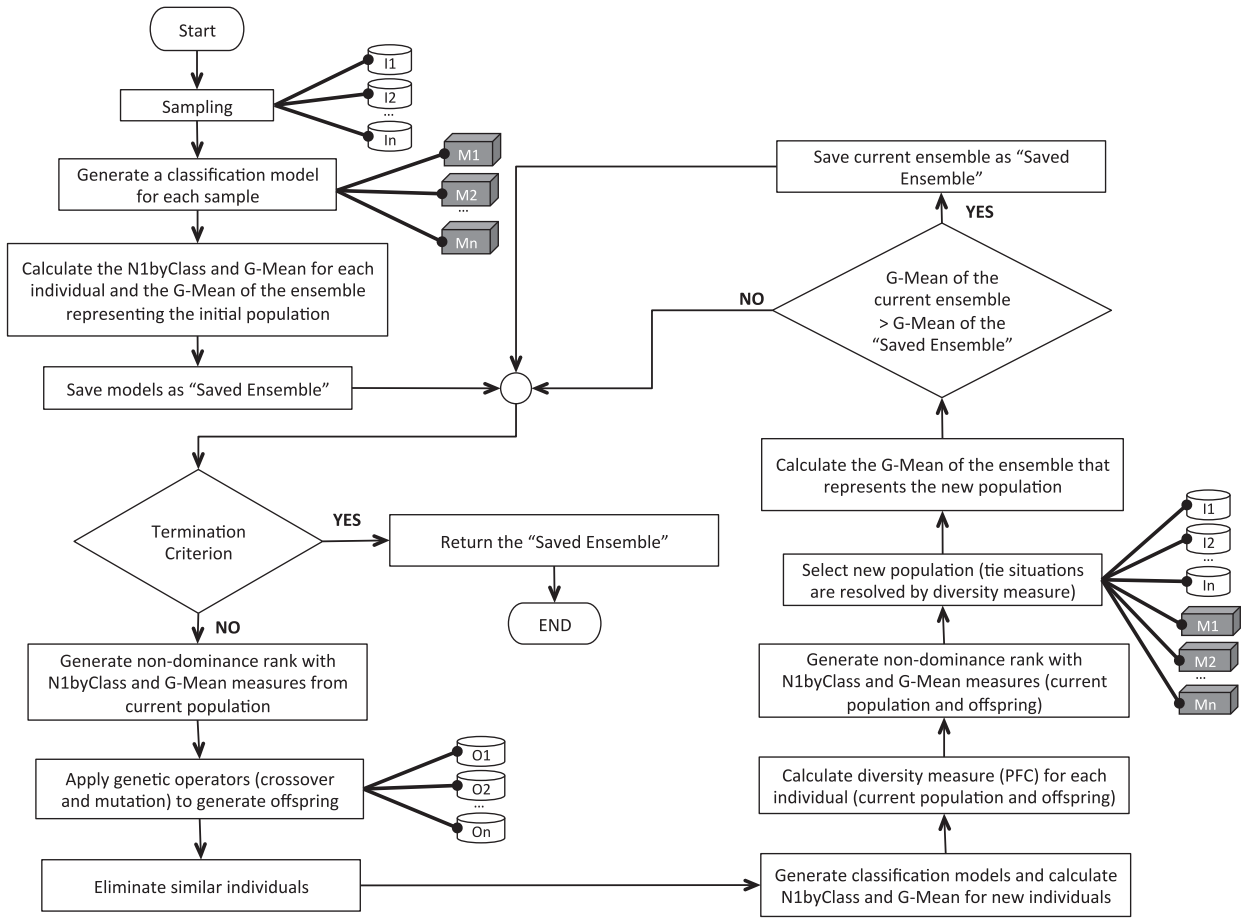


Fig. 2. EVINCI's Workflow.

Classes															
Parent 1	1	0	1	1	0	0	1	1	1	0	1	0	0	0	1
Parent 2	0	1	1	0	0	1	0	1	1	0	1	1	1	0	0
Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

(a) Vectors representing random samples of a dataset with 15 instances and 3

Class						Offspring 1					
Parent 1						Offspring 1					
Parent 2						Offspring 2					
Index	1	5	8	10	13	Index	1	5	8	10	13

(b) One-point crossover per class

Fig. 3. Crossover process.

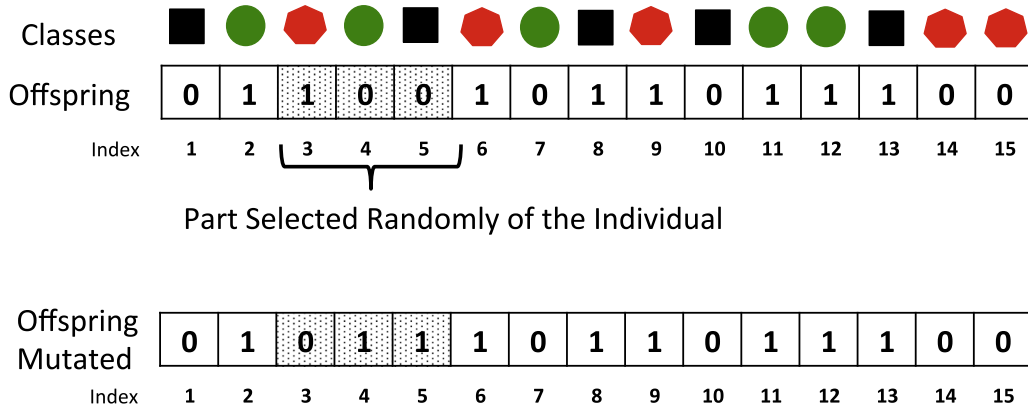


Fig. 4. Mutation process.

better levels of non-dominance are selected (i.e. non-dominance rank equal to 0), then only those who are not dominated by the first individuals, and so on until the default population size is reached. Tie situations are resolved using the measure of classifier diversity PFC.

As noted earlier, the proposed method uses the PFC measure of classifier diversity, due to its good results in imbalanced classification and because it shows more compliance with the performed search than the crowding distance measure used by NSGA-II does. This is because the crowding distance is calculated taking into account the values of the objectives used in the evolutionary algorithm, giving preference to solutions that are more distant from the others in the objective space. However, the PFC indicates the diversity of the classification model associated with an individual in relation to the other models of the population and we are looking for more diverse classification models, aiming at constructing an effective ensemble of classifiers. PFC is measured for each individual based on its pair-wise comparison with all individuals in the intermediary population.

The composition of the ensemble is meant to relieve the loss of information inherent in the sampling process. Thus, different classifiers should have different views of the dataset. This is supported by the diversity mechanisms we incorporated into the proposed method. Despite the individuals selected for the next generation having fitness values better than or equal to the individuals of the current generation, this does not guarantee that the resulting ensemble of these individuals will be more accurate than the ensemble of the current generation. For this reason, in the initial population and after each generation, the classification models of all individuals in the current generation comprise an ensemble of classifiers representing the generation. This ensemble is evaluated based on the entire training dataset and the G-mean accuracy measure is extracted from this evaluation. First, the models of the initial population and its G-mean are saved as “*Saved Ensemble*”. After each generation, the G-mean of the current ensemble population is compared with that of the “*Saved Ensemble*”. If the current ensemble of classifiers shows improvement in the G-mean, the models of the current population replace the “*Saved Ensemble*”.

This process stops after a fixed number of generations, after 10 generations without any replacement of the “*Saved Ensemble*”, or when the G-mean of the ensemble reaches its maximum value, i.e., $\max. G\text{-mean} = 1.0$. The classification models of all individuals in the final “*Saved Ensemble*” comprise the ensemble of classifiers returned by EVINCI. When a new example is presented to the system, its class is determined by the majority vote based on the output of all the classifiers.

6. Experiments

In this section, we present an empirical analysis of EVINCI, including a comparison of its performance with those of other ensemble-based methods that have been proposed for imbalanced learning. In addition, we analyze the EVINCI results when using only one objective function. That is, we determine how the proposed method behaves when its only objective is to decrease the density of the majority classes in the overlapping areas and how it behaves when using only model accuracy as a selection factor. Our goal in these experiments was to verify whether the proposed method actually offers some advantage in terms of overall performance and to examine its influence on the learning process. The comparisons also enabled us to determine the individual strengths and weaknesses of the proposed method compared to other state-of-art approaches.

6.1. Experimental setup

In the experiments, we included twenty-two datasets with different imbalance ratios and with the number of classes ranging from 2 to 18. We obtained these datasets from the UCI [3] and Keel [1] repositories, except for the *Dnormal* dataset, which is an artificial dataset. All these datasets are summarized in Table 2, including the number of classes (#C), number of features (#F), multi-class imbalance ratio (Imb Ratio) [42], and class distribution.

Table 3

G-mean values achieved by different methods in the experiments over 30 runs with their ranks by dataset (between parentheses), G-mean average for each method, ranking count for each method, and ranking average.

Data Set	EVINCI	SmoteBagging	RUSBagging	ROSBagging	AdaboostM1	Rusboost
Abalone	0.0041 (1)	0.0000 (2)	0.0000 (2)	0.0000 (2)	0.0000 (2)	0.0000 (2)
Balance-scale	0.5435 (3)	0.1466 (6)	0.6130 (2)	0.4914 (4)	0.3145 (5)	0.6204 (1)
Car	0.8274 (4)	0.8318 (3)	0.8099 (5)	0.8795 (1)	0.8551 (2)	0.7591 (6)
Chess	0.5956 (4)	0.6144 (3)	0.3218 (5)	0.6731 (2)	0.6759 (1)	0.2866 (6)
Contraceptive	0.5155 (1)	0.5065 (4)	0.5106 (2)	0.5019 (5)	0.4765 (6)	0.5083 (3)
Dermatology	0.9643 (2)	0.9741 (1)	0.9524 (5)	0.9561 (4)	0.9607 (3)	0.9324 (6)
Dnormal	0.8755 (1)	0.8709 (3)	0.8748 (2)	0.8692 (4)	0.8483 (6)	0.8673 (5)
Ecoli	0.7939 (2)	0.7953 (1)	0.7838 (3)	0.7665 (4)	0.7266 (6)	0.7622 (5)
Ecoli2	0.8620 (3)	0.8574 (5)	0.8659 (2)	0.8581 (4)	0.8439 (6)	0.8708 (1)
Glass	0.6621 (1)	0.5564 (5)	0.5709 (4)	0.6352 (3)	0.6574 (2)	0.4511 (6)
New-thyroid	0.9045 (2)	0.8908 (6)	0.8998 (3)	0.8984 (4)	0.8937 (5)	0.9128 (1)
Nursery	0.9330 (4)	0.9602 (3)	0.9062 (5)	0.9700 (2)	0.9830 (1)	0.8988 (6)
Oilspill	0.7678 (2)	0.6604 (5)	0.7980 (1)	0.7026 (4)	0.6174 (6)	0.7519 (3)
Page-blocks	0.9424 (1)	0.8841 (4)	0.9205 (2)	0.8665 (5)	0.8308 (6)	0.9101 (3)
Penbased	0.9156 (5)	0.9290 (2)	0.9261 (4)	0.9278 (3)	0.9562 (1)	0.8899 (6)
Poker	0.4877 (1)	0.0179 (6)	0.4218 (2)	0.2328 (3)	0.0894 (4)	0.0751 (5)
Satellite	0.8695 (2)	0.8647 (5)	0.8678 (4)	0.8714 (1)	0.8684 (3)	0.8605 (6)
Shuttle	0.9980 (1)	0.9970 (4)	0.9960 (6)	0.9980 (3)	0.9980 (2)	0.9965 (5)
Thyroid	0.9766 (1)	0.9675 (3)	0.9378 (5)	0.9737 (2)	0.8836 (6)	0.9638 (4)
Winequality	0.6611 (2)	0.0872 (6)	0.5929 (3)	0.2132 (5)	0.2994 (4)	0.6701 (1)
Yeast	0.2780 (2)	0.1346 (5)	0.1507 (3)	0.1444 (4)	0.0000 (6)	0.3064 (1)
Yeast5	0.9586 (1)	0.8978 (4)	0.9555 (3)	0.8725 (5)	0.8241 (6)	0.9565 (2)
G-Mean Average	0.7426	0.6566	0.7126	0.6956	0.6638	0.6932
Ranking Count	9-7-2-3-1-0	2-2-5-4-5-4	1-7-5-3-5-1	2-4-4-8-4-0	3-4-2-2-2-9	5-2-3-1-4-7
Ranking Average	2.09	3.91	3.32	3.36	4.05	3.82

Here, we report the results of 30 trials of stratified 5-fold cross-validation. In this procedure, we divided the original dataset into five non-intersecting subsets, each of which maintains the original class imbalance ratio. For each fold, we trained each algorithm used in the experiments using the examples of the remaining fold, and considered the predictive model performance to be the prediction accuracy rate of the induced model tested on the current fold.

Then, we compared the performance of the proposed method with five state-of-art ensemble-based methods that have been proposed in the literature for imbalanced learning. We used the same base classifier and the same size of the resulting ensemble to ensure the fairness of the comparison. It was used the implementation of C4.5 [36] available in the RWeka package [26]. All the methods used in these experiments return an ensemble consisting of ten base classifiers. The ensemble-based methods we used in the experiments included: OverBagging [44], UnderBagging [4], SmoteBagging [44], AdaboostM1 [16], and RUSBoost [40]. We used 20 generations as the stopping criterion in EVINCI, or when the *Saved Ensemble* had reached its maximum G-mean value, i.e., $\max. \text{G-mean} = 1.0$, or after producing 10 generations without observing any improvement in the G-mean from that of the *Saved Ensemble*.

6.2. Experimental results - compared methods

In this section, we present the experimental results obtained for each dataset listed in Table 2. For each dataset, the six methods executed 5-fold cross-validation ten times. Table 3 shows the average G-mean values obtained by each method and their ranking position (in parentheses) for each dataset in the experiment. The last three rows summarize the results comparing the six methods in all the datasets considered. The first row is the *G-Mean Average* obtained by the compared methods. The *Ranking Count* row shows the number of datasets in which each technique obtained the best G-Mean value, the second best, and so on. For example, the six numbers 3-4-2-2-2-9 in the fifth column represent the ranking of the AdaboostM1 method. It obtained the highest G-Mean value on just three of the 22 datasets, the second highest values on four datasets, and so on. The *Average Ranking* row shows the average of the ranking position obtained by each method on all datasets.

Table 3, shows that EVINCI obtained the best G-mean average (0.7426) of all the compared methods. It also received the highest number of best rankings, with nine wins, as well as the lowest average rank of 2.09. The proposed method obtained the first or second best position a total of 16 times, which is twice as many as the method with the second highest G-mean average, i.e., RUSBag. RUSBag obtained an average G-mean score of 0.7126, but obtained the first or second best position result only eight times, with just one at the best position and an average rank of 3.32.

The above results indicate that the proposed method achieved the best overall performance. The ranking provided by the Friedman test supports this assumption, showing EVINCI to be the best-ranked method. The Friedman test also rejects the null-hypothesis, i.e., it says that there is a statistically significant difference between the algorithms ($p\text{-value} = 2.190271 \times 10^{-3}$). Hence, we executed the Nemenyi post-hoc test for pairwise comparison. In our experiments, the proposed method outperformed SmoteBag, AdaboostM1, and Rusboost with a statistical significance at the 95% confidence level.

Table 4

Training time of each of the 6 (six) methods in 3-class artificial datasets ranging in size from 1000 to 10,000 examples and with an imbalance rate maintained at 1.25.

Dataset's Size	EVINCI	SmoteBagging	RUSBagging	ROSBagging	AdaboostM1	Rusboost
1000	15.3387	0.6120	0.3634	0.4206	1.3346	0.9164
2000	46.4650	0.7952	0.3664	0.9660	1.1158	1.2976
3000	97.2164	1.0822	0.5226	1.1038	1.9380	0.9126
4000	163.3650	1.2116	0.4044	0.6198	1.2400	1.0128
5000	290.9789	1.3928	0.3948	1.2098	1.3036	1.0146
6000	519.3870	1.6468	0.4630	0.8466	1.4342	1.0388
7000	562.6550	2.0424	0.4164	0.9666	1.5668	1.1040
8000	715.3922	2.0746	0.4038	1.0956	1.7970	1.1998
9000	935.2278	2.3856	0.4034	1.1988	2.1610	1.2026
10000	1087.8822	2.7552	0.3932	1.3148	2.6140	1.2608

Table 5

G-mean achieved by different versions in the experiments over 30 runs, G-mean Average for each version, ranking count for each version, and ranking average.

Data Set	Two Objectives	One Objective	
	EVINCI	G-Mean	N1byClass
Abalone	0.0041	0.0000	0.0000
Balance-scale	0.5435	0.5022	0.5273
Car	0.8274	0.8214	0.8140
Chess	0.5956	0.5108	0.3891
Contraceptive	0.5155	0.4965	0.5114
Dermatology	0.9643	0.9527	0.9640
Dnormal	0.8755	0.8626	0.8687
Ecoli	0.7939	0.7588	0.7870
Ecoli2	0.8620	0.8395	0.8528
Glass	0.6621	0.6593	0.6676
New-thyroid	0.9045	0.8861	0.8973
Nursery	0.9330	0.9311	0.9220
Oilspill	0.7678	0.7529	0.7776
Page-blocks	0.9424	0.9412	0.9340
Penbased	0.9156	0.9061	0.9069
Poker	0.4877	0.5836	0.3983
Satellite	0.8695	0.8665	0.8712
Shuttle	0.9980	0.9983	0.9979
Thyroid	0.9766	0.9610	0.9689
Winequality	0.6611	0.6273	0.6617
Yeast	0.2780	0.2326	0.2464
Yeast5	0.9586	0.9147	0.9578
G-Mean Average	0.7426	0.7275	0.7237
Ranking Count	16-6-0	2-5-15	4-11-7
Ranking Average	1.27	2.59	2.13

If we add the processing of the meta-heuristic used in the proposed method with the processing required to calculate the measures used to guide its evolutionary process, namely N1byClass, G-mean, and PFC, EVINCI has a higher training time than the other ensemble methods presented in this section. To build an ensemble of 10 (ten) base classifiers, EVINCI analyzes samples and induces, on average, 86 classifiers, against the induction of only 10 (ten) classifiers of the non-evolutionary ensembles. Table 4 shows the training time of each of the 6 (six) methods in 3-class artificial datasets ranging in size from 1000 to 10,000 examples and with an imbalance rate [42] maintained at 1.25. The datasets were generated through the 2dnormal function from the mlbench package [30] available for the R language. The experiments were performed on an Intel Xeon E5-2680 v2 2.8GHz processor with 10 (ten) cores. However, it is worth mentioning that after the training process, the response time of the methods is the same, which is the time required for each classifier to produce its output plus the execution time of the consensus function.

6.3. Further analysis

To analyze whether the aggregation of the two objectives positively influences the proposed method, we conducted experiments on the 22 datasets by running EVINCI with only one objective at a time. Table 5 shows the G-mean average obtained in a 30-time 5-fold cross-validation of these analyses. The proposed method used the two objectives in the first column, only the G-mean as the sampling process objective in the G-mean column, and only the density in the overlapping areas in the N1byClass column.

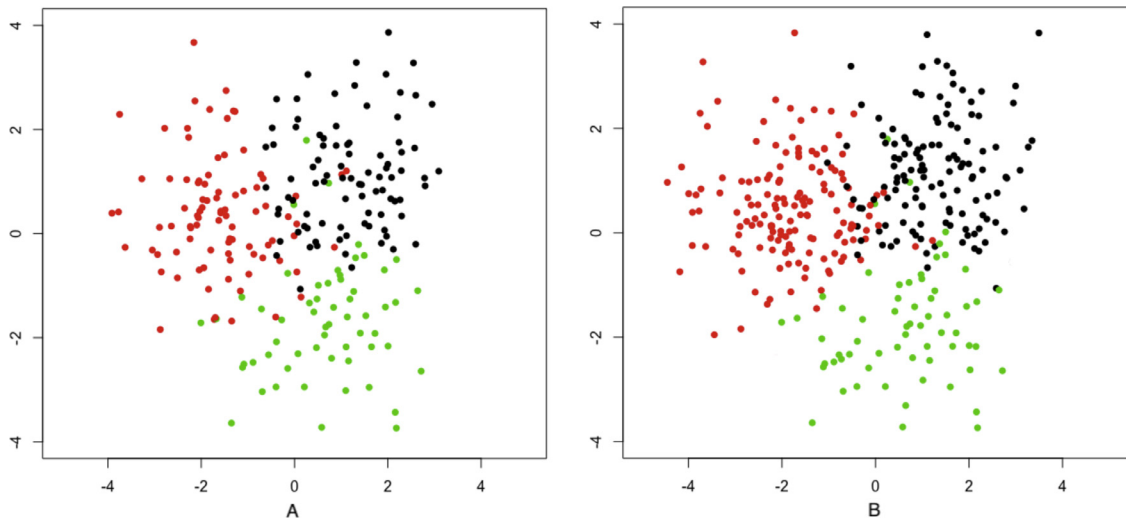


Fig. 5. Fig. A represents a sample taken from initial population and Fig. B a sample from fifth generation.

Table 5, shows that EVINCI, when using both goals, achieved the best predictive results in most of the studied datasets, 16 out of 22. It also obtained the highest average G-mean of 0.7426 versus 0.7275 and 0.7237 when using only the G-mean and the density in the overlapping areas as targets, respectively. The Friedman test supports this result, indicating that the proposed method, when using both objectives, obtained the best classification. This means that there is a statistically significant difference between the algorithms ($p\text{-value} = 2.476668 \times 10^{-6}$). The Nemenyi post-hoc test result indicates that the original EVINCI outperforms the other versions with a statistical significance at the 95% confidence level.

The second position in the Friedman ranking was obtained when using only the density of the majority classes in the regions of overlap as the genetic algorithm's objective. In fact, this version obtained the second position in half of the experimental datasets and obtained the best G-mean value in four of them. As stated previously, in this version the G-mean of the sample-induced model is not used as the objective of the genetic algorithm. However, classifiers are induced to analyze the ensembles of classifiers resulting from the generation. Despite this, the fact that this version obtained the second-best position is a good indication that a set of samples having a higher density of minority classes in the overlap areas can generate an ensemble of classifiers with high predictive performance. In fact, this version of EVINCI obtained a better G-mean average than the second-best ensemble-based method used in the experiments, i.e., RUSBag, which obtained 0.7237 against 0.7126.

As an example of the processing of the proposed method and the evolution of the samples, consider Fig. 5. Fig. 5A shows a sample (individual) randomly taken from the initial population and Fig. 5B shows a random individual from the fifth generation. These are samples from an artificial dataset (*Dnormal* in Table 2), in which green indicates the minority class. As can be observed, the sample shown in Fig. 5A has a low overall imbalance ratio, but has separation boundaries without any particular class dominance. In Fig. 5B, we note that the majority classes had an increased number of instances, but their borders of separation with the minority classes contained more instances of the minority class. A C4.5 classifier induced by sample A obtained accuracy values in the red, black, and green classes, respectively, of 0.9343, 0.8649, and 0.8656 and that by sample B obtained accuracy values of 0.9536, 0.8649, and 0.9402, respectively. This represents an increase in the recognition rate of the minority class. This improvement of accuracy with respect to the minority classes was observed frequently in the other datasets used in the experiments.

7. Conclusion

In this paper, we presented a new evolutionary ensemble-based method for multi-class imbalanced learning, which we called the evolutionary inversion of class distribution for imbalanced learning (EVINCI). Using a MOEA, EVINCI evolves a set of samples taken from an imbalanced dataset to induce an ensemble of classifiers with high predictive accuracy. The evolutionary guidance of the proposed method is based on studies that indicate that the main difficulty experienced by classification algorithms for imbalanced datasets is related to overlapping areas. To address this issue, we developed the data complexity measure N1byClass for use by EVINCI, which produces a matrix of values that estimates the percentage of overlap in each class with other classes.

With the help provided by the N1byClass and the accuracy of the model induced by the samples, EVINCI selectively reduces the concentration of less representative instances of the majority classes in the overlapping areas while selecting samples that produce more accurate models. To increase its generalization power and reduce information loss associated

with the selection process, the EVINCI classification system comprises an ensemble of classifiers, in which each optimized sample induces a different base classifier.

We performed experiments on 22 datasets with different imbalance ratios and numbers of classes ranging from 2 to 18, and the results showed that EVINCI outperforms other relevant methods in most cases. In fact, the proposed method obtained the best G-mean average (0.7426), the highest number of wins (9), and the lowest average rank (2.09). Another interesting point is that, by summing the number of wins and second places obtained by EVINCI in its rank by dataset, the total is twice that obtained by the second-best method, RUSBag.

Further investigation revealed EVINCI's efficiency in the aggregation of the two objective functions, namely, a lower density of majority classes in the overlapping areas and accuracy in the sample-generated model. We observed that the combination of the two objectives enhanced the performance of the proposed method, which obtained the best results in most datasets (16) with a statistical significance of 95%, as determined in a Nemenyi posthoc test.

The version that uses only the density of the overlapping regions obtained the second best result, with a G-mean average higher than that of RUSBag. This is a good indication that a set of samples having a higher density of minority classes in the overlapping areas can generate an ensemble of classifiers with a high predictive performance. In future work, we may consider investigating the possibility of generating such samples without needing to induce classification models during the evolutionary process.

Acknowledgments

The authors would like to thank [FAPESP](#), [CNPq](#), [CAPES](#) and Intel for their financial support. They also would like to thank Sidia Institute of Science and Technology for their support during the conclusion of this work.

References

- [1] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, L. Sánchez, F. Herrera, Keel data-mining software tool: data set repository, integration of algorithms and experimental analysis framework, *J. Mult.-Valued Logic Soft Comput.* 17 (2–3) (2011) 255–287.
- [2] A. Ali, S.M.H. Shamsuddin, A.L. Ralescu, *Classification with Class Imbalance Problem: A Review*, 2015.
- [3] K. Bache, M. Lichman, UCI machine learning repository, 2013. <http://archive.ics.uci.edu/ml>.
- [4] R. Barandela, R. Valdovinos, J. Sánchez, New applications of ensembles of classifiers, *Pattern Anal. Appl.* 6 (3) (2003) 245–256, doi:10.1007/s10044-003-0192-z.
- [5] U. Bhowan, M. Johnston, M. Zhang, X. Yao, Evolving diverse ensembles using genetic programming for classification with unbalanced data, *IEEE Trans. Evol. Comput.* 17 (3) (2013) 368–386.
- [6] L. Breiman, Bagging predictors, *Mach. Learn.* 24 (2) (1996) 123–140, doi:10.1023/A:1018054314350.
- [7] C. Bunkhumpornpat, K. Sinapiromsaran, C. Lursinsap, MUTE: Majority under-sampling technique, in: *2011 8th International Conference on Information, Communications & Signal Processing*, IEEE, 2011, pp. 1–4.
- [8] A. Chandra, X. Yao, Ensemble learning using multi-objective evolutionary algorithms, *J. Math. Model. Algorithms* 5 (4) (2006) 417–445, doi:10.1007/s10852-005-9020-3.
- [9] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, Smote: synthetic minority over-sampling technique, *J. Artif. Intell. Res.* 16 (2002) 321–357.
- [10] S. Das, S. Datta, B.B. Chaudhuri, Handling data irregularities in classification: foundations, trends, and future challenges, *Pattern Recognit.* 81 (2018) 674–693. <http://www.sciencedirect.com/science/article/pii/S0031320318300931>.
- [11] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: nsga-ii, *Trans. Evol. Comp.* 6 (2) (2002) 182–197, doi:10.1109/4235.996017.
- [12] T.G. Dietterich, Machine-learning research – four current directions, *AI Mag.* 18 (1997) 97–136.
- [13] W. Fan, S.J. Stolfo, J. Zhang, P.K. Chan, Adacost: Misclassification cost-sensitive boosting, in: *Proceedings of the Sixteenth International Conference on Machine Learning*, in: *ICML '99*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1999, pp. 97–105. <http://dl.acm.org/citation.cfm?id=645528.657651>.
- [14] E.R.Q. Fernandes, A.C.P.L.F. de Carvalho, A.L.V. Coelho, An evolutionary sampling approach for classification with imbalanced data, in: *Neural Networks (IJCNN), 2015 International Joint Conference on*, IEEE, 2015, pp. 1–7.
- [15] A. Fernández, V. López, M. Galar, M.J. del Jesus, F. Herrera, Analysing the classification of imbalanced data-sets with multiple classes: binarization techniques and ad-hoc approaches, *Knowl. Based Syst.* 42 (2013) 97–110. <http://www.sciencedirect.com/science/article/pii/S0950705113000300>.
- [16] Y. Freund, R.E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting, *J. Comput. Syst. Sci.* 55 (1) (1997) 119–139, doi:10.1006/jcss.1997.1504.
- [17] J.H. Friedman, L.C. Rafsky, Multivariate generalizations of the wald-wolfowitz and smirnov two-sample tests, *Ann. Stat.* 7 (4) (1979) 697–717, doi:10.1214/aos/1176344722.
- [18] M. Galar, A. Fernández, E.B. Tartas, H.B. Sola, F. Herrera, A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches, *IEEE Trans. Systems Man Cybern. Part C* 42 (4) (2012) 463–484.
- [19] V. García, R.A. Mollineda, J.S. Sánchez, On the k-nn performance in a challenging scenario of imbalance and overlapping, *Pattern Anal. Appl.* 11 (3) (2008) 269–280, doi:10.1007/s10044-007-0087-5.
- [20] S. Garca, Z.-L. Zhang, A. Altalhi, S. Alshomrani, F. Herrera, Dynamic ensemble selection for multi-class imbalanced datasets, *Inf. Sci.* 445–446 (2018) 22–37. <http://www.sciencedirect.com/science/article/pii/S0020025518301725>.
- [21] R.L. Graham, P. Hell, On the history of the minimum spanning tree problem, *Ann. Hist. Comput.* 7 (1) (1985) 43–57, doi:10.1109/MAHC.1985.10011.
- [22] L.K. Hansen, P. Salamon, Neural network ensembles, *IEEE Trans. Pattern Anal. Mach. Intell.* 12 (10) (1990) 993–1001, doi:10.1109/34.58871.
- [23] P.E. Hart, The condensed nearest neighbor rule (corresp.), *IEEE Trans. Inf. Theory* 14 (3) (1968) 515–516.
- [24] H. He, Y. Bai, E. García, S. Li, et al., Adasyn: Adaptive synthetic sampling approach for imbalanced learning, in: *Neural Networks, 2008. IJCNN 2008. (IEEE World Congress on Computational Intelligence)*, IEEE International Joint Conference on, IEEE, 2008, pp. 1322–1328.
- [25] T.K. Ho, M. Basu, Complexity measures of supervised classification problems, *IEEE Trans. Pattern Anal. Mach. Intell.* 24 (3) (2002) 289–300, doi:10.1109/34.990132.
- [26] K. Hornik, C. Buchta, A. Zeileis, Open-source machine learning: R meets Weka, *Comput. Stat.* 24 (2) (2009) 225–232, doi:10.1007/s00180-008-0119-7.
- [27] S. Hu, Y. Liang, L. Ma, Y. He, Msmote: Improving classification performance when training data is imbalanced, in: *Computer Science and Engineering, 2009. WCSE '09. Second International Workshop on*, 2, 2009, pp. 13–17, doi:10.1109/WCSE.2009.756.
- [28] T.M. Khoshgoftaar, P. Reboours, Improving software quality prediction by noise filtering techniques, *J. Comput. Sci. Technol.* 22 (3) (2007) 387–396, doi:10.1007/s11390-007-9054-2.

- [29] M. Kubat, S. Matwin, Addressing the curse of imbalanced training sets: One-sided selection, in: *In Proceedings of the Fourteenth International Conference on Machine Learning*, Morgan Kaufmann, 1997, pp. 179–186.
- [30] F. Leisch, E. Dimitriadou, mlbench: Machine Learning Benchmark Problems, 2010. R package version 2.1-1.
- [31] M. Lin, K. Tang, X. Yao, Dynamic sampling approach to training neural networks for multiclass imbalance classification., *IEEE Trans. Neural Netw. Learn. Syst.* 24 (4) (2013) 647–660. <http://dblp.uni-trier.de/db/journals/tnn/tnn24.html#LinTY13>.
- [32] V. López, A. Fernández, S. García, V. Palade, F. Herrera, An insight into classification with imbalanced data: empirical results and current trends on using data intrinsic characteristics, *Inf. Sci.* 250 (Supplement C) (2013) 113–141. <http://www.sciencedirect.com/science/article/pii/S0020025513005124>.
- [33] R. Poli, W.B. Langdon, Genetic programming with one-point crossover, Springer London, London, pp. 180–189. doi:10.1007/978-1-4471-0427-8_20.
- [34] R.C. Prati, G.E.A.P.A. Batista, D.F. Silva, Class imbalance revisited: a new experimental setup to assess the performance of treatment methods, *Knowl. Inf. Syst.* (2014) 1–24. <https://doi.org/10.1007/s10115-014-0794-3>.
- [35] Y. Qian, Y. Liang, M. Li, G. Feng, X. Shi, A resampling ensemble algorithm for classification of imbalance problems, *Neurocomputing* 143 (2014) 57–67.
- [36] R. Quinlan, C4.5: Programs for machine learning, Morgan Kaufmann Publishers, San Mateo, CA, 1993.
- [37] R. Rifkin, A. Klautau, In defense of one-vs-all classification, *J. Mach. Learn. Res.* 5 (2004) 101–141. <http://dl.acm.org/citation.cfm?id=1005332.1005336>.
- [38] J.A. Sáez, J. Luengo, J. Stefanowski, F. Herrera, Smoteipf: addressing the noisy and borderline examples problem in imbalanced classification by a re-sampling method with filtering, *Inf. Sci.* 291 (2015) 184–203. <http://www.sciencedirect.com/science/article/pii/S0020025514008561>.
- [39] B. Schölkopf, J.C. Platt, J. Shawe-Taylor, A.J. Smola, R.C. Williamson, Estimating the support of a high-dimensional distribution., *Neural Comput.* 13 (7) (2001) 1443–1471.
- [40] C. Seiffert, T.M. Khoshgoftaar, J.V. Hulse, A. Napolitano, Rusboost: a hybrid approach to alleviating class imbalance., *IEEE Trans. Syst. Man Cybern. Part A* 40 (1) (2010) 185–197. <http://dblp.uni-trier.de/db/journals/tsmc/tsmca40.html#SeiffertKHN10>.
- [41] Y. Sun, A.K.C. Wong, M.S. Kamel, Classification of imbalanced data: a review, *IJPRAI* 23 (4) (2009) 687–719, doi:10.1142/S0218001409007326.
- [42] A.K. Tanwani, M. Farooq, Classification potential vs. classification accuracy: A Comprehensive study of evolutionary algorithms with biomedical datasets, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 127–144. doi:10.1007/978-3-642-17508-4_9.
- [43] K. Tumer, J. Ghosh, Analysis of decision boundaries in linearly combined neural classifiers, *Pattern Recognit.* 29 (1996) 341–348.
- [44] S. Wang, X. Yao, Diversity analysis on imbalanced data sets by using ensemble models, in: *2009 IEEE Symposium on Computational Intelligence and Data Mining*, 2009, pp. 324–331.
- [45] S. Wang, X. Yao, Multiclass imbalance problems: analysis and potential solutions, *IEEE Trans. Syst. Man. Cybern. Part B (Cybern.)* 42 (4) (2012) 1119–1130, doi:10.1109/TSMCB.2012.2187280.
- [46] Q.-Y. Yin, J.-S. Zhang, C.-X. Zhang, N.-N. Ji, A novel selective ensemble algorithm for imbalanced data classification based on exploratory undersampling, *Math. Probl. Eng.* 2014 (2014) 1–14.